

Índice general

I Creación de preguntas de opción múltiple	2
1. Clase Marcador	4
1.1. Funcionamiento General	4
1.2. Implementación	4
1.2.1. Método <code>__init__(self, data)</code>	5
1.2.2. Método <code>__iter__(self)</code>	5
1.2.3. Método <code>__next__</code>	5
2. Clases Supuesto y Cuestion	7
2.1. Clase <code>Supuesto(enunciado, semantica, precondition)</code>	7
2.2. Clase <code>Cuestion(enunciado, semantica, precondition)</code>	7
3. Clase ProblemaTipo	8
3.1. Implementación	8
3.1.1. Método <code>__iter__(self)</code>	8
3.1.2. Método <code>__next__</code>	9
3.2. Ejemplo de funcionamiento	10
4. Clase ProblemaTipoProfe	14
4.1. Implementación	14
4.1.1. Método <code>__next__</code> para la clase <code>ProblemaTipoProfe</code>	14
4.1.2. Función auxiliar <code>CuestionesJuntas</code>	15
4.2. Ejemplo de funcionamiento	15
5. Clase ProblemaVF	17
5.1. Implementación	17
5.1.1. Métodos de la clase <code>ProblemaVF</code>	17
5.2. Ejemplo de funcionamiento	18
5.2.1. Generación de cuatro variantes cortas	18
5.2.2. Generación de cinco variantes con presentación formateada	18
6. ¿Y ahora qué?	20
6.1. ¿Qué tenemos?	20
6.2. ¿Qué nos falta?	20

Parte I

Creación de preguntas de opción múltiple

Estructura del paquete calccprop-quiz

El módulo completo está compuesto varios fragmentos de código se son descritos en los siguientes capítulos.

```
----- calccprop_quiz/__init__.py -----  
from calccprop import *
```

```
<<Metodo auxiliar para juntar cuestiones en formato para profe>>  
<<Definición de la clase Marcador>>  
<<Definición de la clase Supuesto>>  
<<Definición de la clase Cuestion>>  
<<Definición de la clase ProblemaTipo>>  
<<Definición de la clase ProblemaTipoProfe>>  
<<Definición de la clase ProblemaVF>>
```

Uso del módulo

Para usar este módulo, primero se debe ejecutar

```
from calccprop_quiz import *
```

Capítulo 1

Clase Marcador

Usaremos la clase `Marcador` como una herramienta auxiliar para construir el árbol con todas las combinaciones posibles entre un conjunto de proposiciones y textos (que componen un conjunto de posibles enunciados), y un conjunto de cuestiones relacionadas con dichos enunciados.

(Posteriormente se descartarán de dicho árbol las combinaciones enunciado-cuestión en las que la veracidad o falsedad de la cuestión no se pueda deducir con las proposiciones declaradas en el enunciado.)

1.1. Funcionamiento General

Invocamos esta clase del siguiente modo: `Marcador(data)`, donde `data` es una lista de números enteros mayores que cero. Por ejemplo, si invocamos `Marcador([2, 3, 1])`, estamos pidiendo todas las listas `[i, j, k]` tales que $0 \leq i < 2$, $0 \leq j < 3$ y $0 \leq k < 1$ (es decir, la primera componente solo puede tomar dos valores (0 y 1), la segunda tres (0, 1 y 2) y la tercera solo puede valer 0).

```
for i in Marcador([2,3,1]):  
    print(i)
```

```
[0, 0, 0]  
[0, 1, 0]  
[0, 2, 0]  
[1, 0, 0]  
[1, 1, 0]  
[1, 2, 0]
```

1.2. Implementación

La clase `Marcador` es un [iterador](#). Para el argumento `data=[n_1, ..., n_k]` genera todas las listas de la forma `[m_1, ..., m_k]` tales que cada `m_i` es un número natural menor que `n_i`.

Esta clase `Marcador` está formada por los siguientes fragmentos de código:

```
class Marcador:  
    <<Método __init__ para la clase Marcador>>  
    <<Método __iter__ para la clase Marcador>>  
    <<Método __next__ para la clase Marcador>>
```

__init__ El método `__init__` es el inicializador de instancias en Python y actúa como el constructor de la clase; su función principal es configurar el estado inicial de un objeto recién creado asignándole valores a sus atributos mediante el argumento `self`. Este método especial se ejecuta de forma automática y obligatoria cada vez que se instancia una clase, permitiendo que cada objeto nazca con datos propios y personalizados, lo que evita tener que definir las variables de manera manual tras la creación del objeto. Para profundizar en los detalles técnicos de su funcionamiento, la herencia y las buenas prácticas, puede consultar la sección sobre clases en la [Documentación Oficial de Python](#).

Además, para construir un iterador necesitamos los métodos `__iter__` y `__next__`.

__iter__ El método especial **__iter__** es el encargado de hacer que un objeto sea iterable en Python, permitiendo que pueda ser utilizado directamente en bucles **for**, comprensiones de listas o cualquier contexto que requiera recorrer elementos uno a uno. Su papel preciso es devolver un objeto iterador (que puede ser el propio objeto **self** si este implementa el método **__next__**, o un objeto iterador dedicado de otra clase), estableciendo así el protocolo de iteración de Python. Al invocar este método, Python sabe exactamente cómo comenzar el recorrido de la estructura de datos, ya sea una colección nativa como una lista o una clase personalizada que hayas diseñado. Para comprender a fondo cómo implementar este método junto con el protocolo de iteración y los generadores, puede consultar la sección sobre iteradores en la [Documentación Oficial de Python](#).

__next__ El método especial **__next__** es la pieza fundamental que hace funcionar a un objeto **iterador** en Python, siendo el encargado de devolver el siguiente elemento disponible cada vez que se avanza en una secuencia. Este método es invocado automáticamente entre bastidores por la función nativa **next()** o al recorrer un bucle, y tiene la responsabilidad crucial de lanzar la excepción estándar **StopIteration** cuando se han agotado todos los elementos, lo que le indica a Python que la iteración ha terminado de forma segura. Junto con **__iter__**, conforma el protocolo de iteración completo de Python, permitiendo un control milimétrico sobre cómo se calculan y entregan los datos (incluso bajo demanda o de forma infinita). Para ver ejemplos prácticos de su implementación y entender cómo gestiona el flujo de datos, puede revisar la documentación sobre tipos iteradores en la [Referencia de la Biblioteca Estándar de Python](#).

1.2.1. Método **__init__**(self, data)

Este método inicializa la clase y almacena en el atributo **self.data** el argumento **data**; que es una lista con los topes de cada una de las componentes de las listas que se van a crear. En el ejemplo anterior, **[2,3,1]**.

```
def __init__(self, data):  
    self.data = data
```

1.2.2. Método **__iter__**(self)

El método **__iter__** inicializa el estado del iterador. Este método genera la semilla del iterador. Dicha semilla es una lista de ceros (con tantos ceros como elementos hay en **self.data**). La lista **self.p** se entregará a **__next__** en la siguiente invocación al método.

```
def __iter__(self):  
    self.p = [0 for x in self.data]  
    return self
```

1.2.3. Método **__next__**

Gestiona el avance del iterador.

- **__next__** devuelve **self.p** en su estado actual (**n**), y coloca en **self.p** la lista siguiente usando la función auxiliar **Siguiente**.
- Si **self.p** es vacía detiene la iteración.

```
def __next__(self):  
    <<Definición de la función local Siguiente>>  
    if self.p == []:  
        raise StopIteration  
    n = self.p  
    self.p = Siguiente(self.p, self.data)  
    return n
```

1. Función auxiliar **Siguiente** (x, y)

Dentro del método **__next__** definimos de manera recursiva la función auxiliar local **Siguiente**, que es la función que calcula la lista siguiente.

La función **Siguiente(x, y)** es el corazón lógico de la clase. Utiliza **recursividad** para implementar un orden lexicográfico:

- x : lista de números naturales $[m_1 \dots m_r]$ tales que $m_i < n_i$ (es la lista de la que queremos calcular la siguiente).
- y : lista de números naturales $[n_1 \dots n_r]$ mayores que cero (lista de toques).

La función `Siguiente` devuelve una lista $[k_1 \dots k_r]$; que es la siguiente lista a $[m_1 \dots m_r]$ (según el orden lexicográfico) que aún cumple que $k_i < n_i$. En caso de no existir tal lista devuelve $[]$.

a) Implementación:

- Intenta incrementar primero el último elemento de la lista (el de la derecha).
- Si el elemento de la derecha alcanza su límite ($x[0]+1 == y[0]$), intenta incrementar el elemento anterior y resetea.^a cero todos los elementos a su derecha.
- Si todos los elementos han alcanzado su límite, devuelve una lista vacía $[]$, lo que señala el fin de la iteración.

Definición de la función local `Siguiente`

```
def Siguiente(x,y):
    if x == [] :
        return []
    s = Siguiente(x[1:],y[1:])
    if s == []:
        if x[0]+1 == y[0]:
            return []
        else:
            return [x[0]+1] + [0 for i in x[1:]]
    else:
        return [x[0]] + s
```

Capítulo 2

Clases Supuesto y Cuestion

2.1. Clase Supuesto(enunciado, semantica, precondition)

La clase `Supuesto` define la estructura para almacenar tres aspectos (atributos) del planteamiento de un problema mediante su constructor `__init__`.

enunciado cadena de texto que presenta los supuestos en lenguaje humano.

semantica proposición que traduce parcialmente el significado lógico del **enunciado** (contiene lo necesario para deducir las respuestas).

precond es una proposición o bien cualquiera de los valores `True` o `False`. El objeto `Supuesto` será incluido en el problema solo si la evaluación de **precond** es `True`.

Definición de la clase Supuesto

```
class Supuesto:
    def __init__(self, enunciado, semantica, precondition=True):
        self.e = enunciado
        self.s = semantica
        self.p = precondition

    def __repr__(self):
        """Método de representación"""
        return 'Cuestión( Enunciado: ' + self.e + '; Semántica: ' + self.s + '; Precondición: ' + self.p + ' )'
```

2.2. Clase Cuestion(enunciado, semantica, precondition)

De manera análoga, la clase `Cuestion` define la estructura para almacenar cuatro aspectos (atributos) del planteamiento de un problema mediante su constructor `__init__`.

enunciado string que presenta la pregunta en lenguaje humano.

semantica proposición `P` tal que `test(P, semantica_de_los_supuestos)` devuelve `True` si es imposible refutarlo a partir de las premisas, es decir, es imposible que `P` sea falso si las premisas son verdad.

precond es una proposición o bien cualquiera de los valores `True` o `False`. El objeto `Cuestion` será incluido en el problema solo si la evaluación de **precond** es `True`.

exp al exportar las preguntas al formato `xml` de Moodle, cabe la posibilidad de que Moodle ofrezca al alumno una explicación una vez ha contestado a las preguntas. En el atributo **exp** se especifica la cadena de texto con la explicación que debe mostrar Moodle.

Definición de la clase Cuestion

```
class Cuestion:
    def __init__(self, enunciado, semantica, precondition=True, exp=""):
        self.e = enunciado
        self.s = semantica
        self.p = precondition
        self.x = exp

    def __repr__(self):
        """Método de representación"""
        return 'Cuestión( Enunciado: ' + self.e + '; Semántica: ' + self.s + '; Precondición: ' + self.p + ' )'
```

Capítulo 3

Clase ProblemaTipo

La clase `ProblemaTipo` permite, a partir de una lista de posibles enunciados y varias listas de posibles preguntas, generar muchas variantes de ejercicios de opción múltiple por simple combinatoria. Combina enunciados con una sublista de preguntas y, para cada pregunta calcula su veracidad o falsedad dados los supuestos del enunciado. Si no es posible decidir la veracidad o falsedad de una pregunta para un enunciado concreto, esa combinación enunciado-pregunta es descartada.

3.1. Implementación

La clase `ProblemaTipo` es un iterador que genera todas las versiones aceptables de un problema. Cada variante resulta de tomar una de las opciones de cada una de las listas en `supuestos_y_cuestiones`. Si la precondition de alguna de las opciones elegidas es `False` (o resulta refutable), la variante entera del problema es rechazada y se evalúa la siguiente combinación disponible.

- El atributo `self.e` contiene la lista `supuestos_y_cuestiones`. El argumento `supuestos_y_cuestiones` es una lista de listas de strings, objetos `Supuesto` o objetos `Cuestion`. Cada lista representa un conjunto de opciones excluyentes.

Por tanto, cada elemento de `supuestos_y_cuestiones` es a su vez una lista de opciones (excluyentes) correspondiente a cada componente (cada parte) del texto del problema.

Por comodidad, en el caso de que una de esas listas consista en un único objeto (una única opción), también se permite escribir directamente dicho objeto sin encerrarlo en una lista; es decir, la lista `supuestos_y_cuestiones` también admite directamente objetos de tipo string, `Supuesto` o `Cuestion`.

La combinación de distintas opciones crea el texto completo de una variante del problema.

Definición de la clase `ProblemaTipo`

```
class ProblemaTipo:
    def __init__(self, supuestos_y_cuestiones):
        self.e = supuestos_y_cuestiones

    <Método __iter__ para la clase ProblemaTipo>
    <Método __next__ para la clase ProblemaTipo>
```

El método `__iter__` inicializa el iterador configurando las estructuras internas, mientras que el método `__next__` obtiene la siguiente variante válida del problema, asegurándose de desechar en el proceso aquellas cuyas condiciones no se satisfagan.

3.1.1. Método `__iter__(self)`

El atributo `self.l` es la lista `self.e` normalizada para que todos sus objetos sean listas; `self.long` es el número de componentes que constituyen el texto de cada variante. `self.i` es el iterador interno encargado de la combinatoria y `self.c` es un contador de intentos de generación.

Método `__iter__` para la clase `ProblemaTipo`

```
def __iter__(self):
    self.l = [x if isinstance(x, list) else [x] for x in self.e]
    self.long = len(self.l)
    self.i = iter(Marcador([len(x) for x in self.l]))
    self.c = 0
    return self
```

3.1.2. Método `__next__`

La generación de variantes de un `ProblemaTipo` se produce dentro de un bucle que solo se detiene cuando el intento de iterar el marcador interno, `next(self.i)`, lanza una excepción `StopIteration` por haberse agotado todas las combinaciones posibles.

En cada intento de generación:

1. El contador `self.c` se incrementa de forma consecutiva.
2. Las variables `enunciado`, `hipotesis` y `cuestiones` se inicializan vacías.
3. Se recorren uno a uno los componentes de la variante candidata.

Si todos los componentes superan con éxito sus respectivas validaciones, el método devuelve una tupla con el identificador del intento y las estructuras generadas: `(str(self.c), enunciado, cuestiones)`.

Método `__next__` para la clase `ProblemaTipo`

```
def __next__(self):
    self.c += 1
    while True:
        try:
            variante = next(self.i)
        except StopIteration:
            raise StopIteration

        enunciado = ""
        hipotesis = []
        cuestiones = []

        for n in range(self.long+1):
            if n == self.long:
                return (str(self.c), enunciado, cuestiones)

        componente = self.l[n][variante[n]]
        <<Tratamiento en función del tipo de componente>>
```

El fragmento de código `<<Tratamiento en función del tipo de componente>>` procesa el componente que corresponda en cada iteración del bucle `for` según su tipo:

- Cuando el `componente` es una cadena, se considera parte del enunciado del problema y se concatena al string `enunciado`.
- Cuando el `componente` es un objeto `Supuesto`, se testea su precondition. Si `componente.p` es `True` o no es refutable a partir de las `hipotesis` acumuladas, su texto `componente.e` se añade al enunciado y su significado lógico `componente.s` se incorpora a las `hipotesis`. En caso contrario, el análisis de la combinación actual se interrumpe de inmediato mediante un `break`, se notifica en pantalla el descarte y se pasa a evaluar el siguiente elemento del marcador.
- Cuando el `componente` es un objeto `Cuestion`, se testea su precondition. Si es `True` o no es refutable a partir de las `hipotesis` acumuladas, el par (`componente.e`, veracidad o falsedad de `componente.s`) se añade a la lista de `cuestiones`. En caso contrario, se imprime la notificación en pantalla y se interrumpe el procesamiento de la variante mediante un `break`; esto hace que la combinación entera quede descartada de inmediato, ignorando cualquier componente o lista de cuestiones posterior que restase por evaluar en esa iteración.

Tratamiento en función del tipo de componente

```
if isinstance(componente, str):
    enunciado = enunciado + componente

elif isinstance(componente, Supuesto):
    if test(componente.p, hipotesis):
        enunciado = enunciado + componente.e
        hipotesis = hipotesis + [componente.s]
    else:
        print('\n Supuesto: ' + str(componente.e) \
              + ' rechazado por ' + str(componente.p) + '\n')
        break

elif isinstance(componente, Cuestion):
    if test(componente.p, hipotesis):
        cuestiones = cuestiones + \
            [(componente.e, (True if test(componente.s, hipotesis) else False), 1, componente.x)]
    else:
        cuestiones = cuestiones + \
            [(componente.e, 'rechazada por ' + str(componente.p), 0, componente.x)]
        print('\n Cuestion: ' + str(componente.e) \
              + ' rechazada por ' + str(componente.p) + '\n')
        break
```

3.2. Ejemplo de funcionamiento

Escribamos un hipotético ejercicio (es decir una lista que contiene cadenas de caracteres, supuestos, cuestiones, sublistas de supuestos o sublistas de cuestiones):

```
ejercicio = [ "Considerare ",
[
    Supuesto("A ", v("A")),
    Supuesto("B ", v("B")),
],
[
    Supuesto("y que A implica C. ", v("A") >> v("C")),
    Supuesto("y que B equivale a D. ", v("B") ** v("D")),
],
"Conteste: ",
[
    Cuestion("¿Se da C?", v("C"), exp="Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A"),
    Cuestion("¿Se da D?", v("D")),
    Cuestion("¿Se da E?", v("E"), v("D")),
],
[
    Cuestion("¿Se dan A y C?", v("A") & v("C")),
    Cuestion("¿Se dan C y D?", v("C") & v("D")),
],
]
```

A continuación veamos lo que hace `ProblemaTipo` con la lista `ejercicio`. Observe que en cada iteración se toma un elemento de cada lista de supuestos y un elemento de cada lista de cuestiones para formar la combinación de un enunciado (compuesto por supuestos) y un subconjunto de preguntas (tantas como sublistas de cuestiones). Si alguna cuestión es descartada en una combinación, se salta a la siguiente combinación (indicando la cuestión descartada y el motivo). Así, `ProblemaTipo` muestra todas las combinaciones posibles de cuestiones que se pueden responder para cada combinación de supuestos. Fíjese que los elementos que no son listas (aquí las dos cadenas de caracteres) funcionan como sublistas con un único elemento (por eso en todas las iteraciones aparece tanto 'Considerare' como 'Conteste:').

```
for i in ProblemaTipo( ejercicio ):
    print(i)
```

```
('1', 'Considerare A y que A implica C. Conteste: ', [(('¿Se da C?', True, 1, 'Solo sabemos que C es cierto y si A implica C y
('2', 'Considerare A y que A implica C. Conteste: ', [(('¿Se da C?', True, 1, 'Solo sabemos que C es cierto y si A implica C y
('3', 'Considerare A y que A implica C. Conteste: ', [(('¿Se da D?', False, 1, ''), ('¿Se dan A y C?', True, 1, ''))])
('4', 'Considerare A y que A implica C. Conteste: ', [(('¿Se da D?', False, 1, ''), ('¿Se dan C y D?', False, 1, ''))])
```

Question: ¿Se da E? rechazada por v('D')

Question: ¿Se da E? rechazada por v('D')

```
('5', 'Considerare A y que B equivale a D. Conteste: ', [(('¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica
('6', 'Considerare A y que B equivale a D. Conteste: ', [(('¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica
('7', 'Considerare A y que B equivale a D. Conteste: ', [(('¿Se da D?', False, 1, ''), ('¿Se dan A y C?', False, 1, ''))])
('8', 'Considerare A y que B equivale a D. Conteste: ', [(('¿Se da D?', False, 1, ''), ('¿Se dan C y D?', False, 1, ''))])
```

Question: ¿Se da E? rechazada por v('D')

Question: ¿Se da E? rechazada por v('D')

```
('9', 'Considerare B y que A implica C. Conteste: ', [(('¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C y
('10', 'Considerare B y que A implica C. Conteste: ', [(('¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C
('11', 'Considerare B y que A implica C. Conteste: ', [(('¿Se da D?', False, 1, ''), ('¿Se dan A y C?', False, 1, ''))])
('12', 'Considerare B y que A implica C. Conteste: ', [(('¿Se da D?', False, 1, ''), ('¿Se dan C y D?', False, 1, ''))])
```

Question: ¿Se da E? rechazada por v('D')

Question: ¿Se da E? rechazada por v('D')

```
('13', 'Considerare B y que B equivale a D. Conteste: ', [(('¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica
('14', 'Considerare B y que B equivale a D. Conteste: ', [(('¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica
('15', 'Considerare B y que B equivale a D. Conteste: ', [(('¿Se da D?', True, 1, ''), ('¿Se dan A y C?', False, 1, ''))])
```

```
(16', 'Considere B y que B equivale a D. Conteste: ', [(¿Se da D?', True, 1, ''), (¿Se dan C y D?', False, 1, '')])
(17', 'Considere B y que B equivale a D. Conteste: ', [(¿Se da E?', False, 1, ''), (¿Se dan A y C?', False, 1, '')])
(18', 'Considere B y que B equivale a D. Conteste: ', [(¿Se da E?', False, 1, ''), (¿Se dan C y D?', False, 1, '')])
```

Cuando la lista de cuestiones es extensa (o incluye aclaraciones), la representación en una tupla puede extenderse más allá de los márgenes de este documento al utilizar `print`. Para mejorar la claridad y facilitar la comprensión del funcionamiento de `ProblemaTipo`, desempaquetaremos los componentes devueltos por el iterador y los presentaremos de forma estructurada. Esto permitirá visualizar mejor los resultados.

En cada iteración, presentaremos el enunciado completo correspondiente, junto con las cuestiones incluidas, cada una en una línea separada (indicando su veracidad y la explicación incluida en el atributo `self.x` si no la hemos dejado vacía). Si la variante fue abortada, se indicará cuál fue la cuestión desencadenante y el motivo de la interrupción.

```
for i in ProblemaTipo( ejercicio ):
    # Desempaquetamos la tupla en variables
    id_pregunta, EnunciadoCompleto, lista_Cuestiones = i

    # Imprimimos los primeros datos
    print(f"ID: {id_pregunta}")
    print(f"Enunciado completo: {EnunciadoCompleto}")

    # Imprimimos la lista de cuestiones, cada cuestión en una línea
    print(*lista_Cuestiones, sep='\n')
    print('\n')
```

```
ID: 1
Enunciado completo: Considere A y que A implica C. Conteste:
(¿Se da C?', True, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')
(¿Se dan A y C?', True, 1, '')
```

```
ID: 2
Enunciado completo: Considere A y que A implica C. Conteste:
(¿Se da C?', True, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')
(¿Se dan C y D?', False, 1, '')
```

```
ID: 3
Enunciado completo: Considere A y que A implica C. Conteste:
(¿Se da D?', False, 1, '')
(¿Se dan A y C?', True, 1, '')
```

```
ID: 4
Enunciado completo: Considere A y que A implica C. Conteste:
(¿Se da D?', False, 1, '')
(¿Se dan C y D?', False, 1, '')
```

```
Question: ¿Se da E? rechazada por v('D')
```

```
Question: ¿Se da E? rechazada por v('D')
```

```
ID: 5
Enunciado completo: Considere A y que B equivale a D. Conteste:
(¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')
(¿Se dan A y C?', False, 1, '')
```

```
ID: 6
Enunciado completo: Considere A y que B equivale a D. Conteste:
(¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')
(¿Se dan C y D?', False, 1, '')
```

```
ID: 7
Enunciado completo: Considere A y que B equivale a D. Conteste:
(¿Se da D?', False, 1, '')
```

('¿Se dan A y C?', False, 1, '')

ID: 8

Enunciado completo: Considere A y que B equivale a D. Conteste:

('¿Se da D?', False, 1, '')

('¿Se dan C y D?', False, 1, '')

Question: ¿Se da E? rechazada por v('D')

Question: ¿Se da E? rechazada por v('D')

ID: 9

Enunciado completo: Considere B y que A implica C. Conteste:

('¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')

('¿Se dan A y C?', False, 1, '')

ID: 10

Enunciado completo: Considere B y que A implica C. Conteste:

('¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')

('¿Se dan C y D?', False, 1, '')

ID: 11

Enunciado completo: Considere B y que A implica C. Conteste:

('¿Se da D?', False, 1, '')

('¿Se dan A y C?', False, 1, '')

ID: 12

Enunciado completo: Considere B y que A implica C. Conteste:

('¿Se da D?', False, 1, '')

('¿Se dan C y D?', False, 1, '')

Question: ¿Se da E? rechazada por v('D')

Question: ¿Se da E? rechazada por v('D')

ID: 13

Enunciado completo: Considere B y que B equivale a D. Conteste:

('¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')

('¿Se dan A y C?', False, 1, '')

ID: 14

Enunciado completo: Considere B y que B equivale a D. Conteste:

('¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')

('¿Se dan C y D?', False, 1, '')

ID: 15

Enunciado completo: Considere B y que B equivale a D. Conteste:

('¿Se da D?', True, 1, '')

('¿Se dan A y C?', False, 1, '')

ID: 16

Enunciado completo: Considere B y que B equivale a D. Conteste:

('¿Se da D?', True, 1, '')

('¿Se dan C y D?', False, 1, '')

ID: 17

Enunciado completo: Considere B y que B equivale a D. Conteste:

('¿Se da E?', False, 1, '')

('¿Se dan A y C?', False, 1, '')

ID: 18

Enunciado completo: Considere B y que B equivale a D. Conteste:

('¿Se da E?', False, 1, '')

('¿Se dan C y D?', False, 1, '')

En el ejemplo anterior, se observa que, para cada combinación de supuestos, en cada iteración solo se presenta una cuestión de cada sublista de cuestiones. Dado que hay solo dos sublistas, esto resulta en la visualización de dos preguntas por iteración, esto dificulta la revisión y mantenimiento del banco de preguntas.

En el anterior ejemplo, y aunque existen únicamente cuatro enunciados posibles y cinco preguntas en total, la combinatoria ha generado 18 variantes no descartables, cada una con un subconjunto de solo dos preguntas. Esta complejidad complica la tarea de revisar y mantener un banco de ejercicios eficaz.

Capítulo 4

Clase ProblemaTipoProfe

Para facilitar la revisión y el mantenimiento de cada ejercicio, es preferible poder ver, junto a cada enunciado, el conjunto completo de cuestiones. Esta es la finalidad de la clase `ProblemaTipoProfe`, que presenta todas las cuestiones para cada enunciado, indicando en cada caso si son `True` o `False` (o si serán descartadas). Para alcanzar este objetivo se utiliza el procedimiento auxiliar `CuestionesJuntas`, para crear una sublista para cada `Cuestion` que incluye únicamente dicha cuestión. De este modo, como al iterar se toma un elemento de cada una de las sublistas, se mostrarán todas las cuestiones a la vez.

4.1. Implementación

Tan solo hay dos diferencias en el código de la clase `ProblemaTipoProfe` respecto al código de la clase `ProblemaTipo`.

- La primera es que en `self.e` cada `Cuestion` aparece como único elemento de una sublista, para así lograr que se ven todas las cuestiones en cada iteración.
- La segunda diferencia se produce en la definición del método `__next__`, que aquí no oculta las cuestiones que sí son rechazadas cuando usamos `ProblemaTipo`.

```
class ProblemaTipoProfe:
    def __init__(self, supuestos_y_cuestiones):
        self.e = CuestionesJuntas(supuestos_y_cuestiones)

    <<Método __iter__ para la clase ProblemaTipo>>
    <<Método __next__ para la clase ProblemaTipoProfe>>
```

4.1.1. Método `__next__` para la clase `ProblemaTipoProfe`

Aquí la diferencia respecto del `<<Método __next__ para la clase ProblemaTipo>>` es el tratamiento de cada componente, pues aquí no se excluyen las preguntas que serán rechazadas en la versión para el estudiante. Aquí todas las cuestiones son incluidas, pero se da una indicación añadiendo 'rechazada por' más la precondición que ocasiona el rechazo cuando se usa `ProblemaTipo`.

```
def __next__(self):
    self.c += 1
    while True:
        try:
            variante = next(self.i)
        except StopIteration:
            raise StopIteration

        enunciado = ""
        hipotesis = []
        cuestiones = []

        for n in range(self.long+1):
            if n == self.long:
                return (str(self.c), enunciado, cuestiones)

        componente = self.l[n][variante[n]]
    <<Tratamiento en función del tipo de componente para la versión del Profe>>
```

Tratamiento en función del tipo de componente en la versión del Profe

Tratamiento en función del tipo de componente para la versión del Profe

```
if isinstance(componente, str):
    enunciado = enunciado + componente

elif isinstance(componente, Supuesto):
    if test(componente.p, hipotesis):
        enunciado = enunciado + componente.e
        hipotesis = hipotesis + [componente.s]
    else:
        print('\n Supuesto: ' + str(componente.e) \
              + ' rechazado por ' + str(componente.p) + '\n')
        break

elif isinstance(componente, Cuestion):
    if test(componente.p, hipotesis):
        cuestiones = cuestiones + \
            [(componente.e, (True if test(componente.s, hipotesis) else False), 1, componente.x)]
    else:
        cuestiones = cuestiones + \
            [(componente.e, 'rechazada por ' + str(componente.p), 0)]
```

4.1.2. Función auxiliar CuestionesJuntas

Este bloque de código define la función `CuestionesJuntas(lista)`, cuyo objetivo principal es **aislar cada objeto de la clase `Cuestion` dentro de su propia sublista individual**, manteniendo otros tipos de datos (como cadenas o listas de supuestos) intactos.

En detalle:

1. **CreaLista(t)**: Una función auxiliar que asegura que cualquier elemento sea tratado como una lista (si no lo es, lo envuelve en una para poder indexarlo).
2. **Iteración**: Recorre cada elemento `e` de la lista de entrada `lista`.
3. **Strings y No-Cuestiones**: Si el elemento es una cadena o si el primer elemento de su contenido (tras pasarlo por `CreaLista`) no es de tipo `Cuestion` (por ejemplo, un objeto de la clase `Supuesto`), se añade directamente a la lista resultante `p` mediante `append`.
4. **Aislamiento de Cuestion**: Si el elemento es una instancia de `Cuestion` o pertenece a una lista de cuestiones, se utiliza `extend(CreaLista(e))`, lo que descompone cualquier agrupación previa de `cuestiones`, añadiendo cada `Cuestion` como una sublista independiente (y con un único elemento) dentro de `p`.

En resumen: Transforma una lista mixta de modo que cada objeto `Cuestion` acabe confinado en una sublista propia. De este modo, el generador multidimensional las procesa de forma independiente y simultánea.

Objetivo: Esto permite que la clase `ProblemaTipoProfe` muestre todas las cuestiones posibles para cada enunciado en lugar de una sola.

Metodo auxiliar para juntar cuestiones en formato para prof e

```
def CuestionesJuntas(lista):
    def CreaLista(t):
        return t if isinstance(t, list) else [t]
    p = []
    for e in lista:
        if isinstance(e, str):
            p.append(e)
        elif not isinstance(CreaLista(e)[0], Cuestion):
            p.append(e)
        else:
            p.extend(CreaLista(e))
    return p
```

4.2. Ejemplo de funcionamiento

Usaremos la misma lista `ejercicio` que usamos en el capítulo anterior

Ejemplo de `ProblemaTipoProfe`

```
for i in ProblemaTipoProfe( ejercicio ):
    print(i)
```

```
('1', 'Considere A y que A implica C. Conteste: ', [( '¿Se da C?', True, 1, 'Solo sabemos que C es cierto y si A implica C y
(2', 'Considere A y que B equivale a D. Conteste: ', [( '¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica
(3', 'Considere B y que A implica C. Conteste: ', [( '¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C y
(4', 'Considere B y que B equivale a D. Conteste: ', [( '¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica
```

Como la vez anterior la representación en una tupla usando `print` se ha extendido más allá de los márgenes de este documento. Para evitar este problema y que el lector vea mejor el funcionamiento de `ProblemaTipoProfe`, vamos a usar el mismo truco que en el capítulo anterior: Debajo de cada variante del enunciado completo, mostraremos todas las cuestiones de manera individual, cada una en una línea separada, junto con el cálculo de su veracidad o la indicación de si serán descartadas al emplear la clase `ProblemaTipo`. Si la cuestión incluye el atributo `self.x` con una explicación, también se muestra la explicación (si no, aparece una cadena vacía `' '`).

```
for i in ProblemaTipoProfe( ejercicio ):
    # Desempaquetamos la tupla en variables
    id_pregunta, EnunciadoCompleto, lista_Cuestiones = i

    # Imprimimos los primeros datos
    print(f"ID: {id_pregunta}")
    print(f"Enunciado completo: {EnunciadoCompleto}")

    # Imprimimos la lista de cuestiones, cada cuestión en una línea
    print(*lista_Cuestiones, sep='\n')
    print('\n')
```

ID: 1

Enunciado completo: Considere A y que A implica C. Conteste:

```
('¿Se da C?', True, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')
('¿Se da D?', False, 1, '')
('¿Se da E?', "rechazada por v('D')", 0)
('¿Se dan A y C?', True, 1, '')
('¿Se dan C y D?', False, 1, '')
```

ID: 2

Enunciado completo: Considere A y que B equivale a D. Conteste:

```
('¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')
('¿Se da D?', False, 1, '')
('¿Se da E?', "rechazada por v('D')", 0)
('¿Se dan A y C?', False, 1, '')
('¿Se dan C y D?', False, 1, '')
```

ID: 3

Enunciado completo: Considere B y que A implica C. Conteste:

```
('¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')
('¿Se da D?', False, 1, '')
('¿Se da E?', "rechazada por v('D')", 0)
('¿Se dan A y C?', False, 1, '')
('¿Se dan C y D?', False, 1, '')
```

ID: 4

Enunciado completo: Considere B y que B equivale a D. Conteste:

```
('¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')
('¿Se da D?', True, 1, '')
('¿Se da E?', False, 1, '')
('¿Se dan A y C?', False, 1, '')
('¿Se dan C y D?', False, 1, '')
```

Vemos que son cuatro enunciados en total, y para cada uno de ellos vemos lo que pasa con cada una de las cinco cuestiones posibles: si son verdaderas, si son falsas o si serán descartadas (y el motivo). Ahora es mucho más fácil mejorar y depurar el `ejercicio` que estamos diseñando para nuestros exámenes de opción múltiple.

Capítulo 5

Clase ProblemaVF

A diferencia de las clases anteriores, que exploran de forma sistemática y secuencial todas las combinaciones posibles de un espacio combinatorio, la clase **ProblemaVF** está diseñada para generar ejercicios basados en un **banco de cuestiones aleatorizado**.

Su objetivo primordial es construir variantes de exámenes del tipo **Verdadero/Falso**, donde el enunciado permanece fijo y cada variante ofrece un subconjunto único y desordenado de preguntas seleccionadas al azar a partir de un repertorio común.

5.1. Implementación

El núcleo de esta clase radica en el uso de la función **sample** del módulo **random** de Python, lo que permite extraer muestras sin repetición del banco de preguntas en cada iteración.

Una particularidad fundamental de este diseño es que el método **__next__** es que no sigue un barrido secuencial de todas las combinaciones posibles. Aquí el iterador generará una variante al azar cada vez que se invoque el método **next()** (obviamente, dado que el banco de cuestiones es finito, aparecerán variantes repetidas de cuando en cuando).

Definición de la clase ProblemaVF

```
from random import sample
class ProblemaVF():
    def __init__(self, enunciado, cuestiones, NumPreguntas):
        self.e = enunciado
        self.c = cuestiones
        self.NumPreguntas = NumPreguntas

    def __iter__(self):
        self.contador = 0
        return self

    def __next__(self):
        cuestiones = sample(self.c, self.NumPreguntas)
        self.contador += 1
        return (str(self.contador), self.e, cuestiones)
```

5.1.1. Métodos de la clase ProblemaVF

- **__init__(self, enunciado, cuestiones, NumPreguntas)**: Inicializa el generador almacenando el texto base del problema (**enunciado**), la lista completa de tuplas con las preguntas y sus valores de verdad (**cuestiones**), y la cantidad exacta de ítems que se deben extraer para cada ejercicio (**NumPreguntas**).
- **__iter__(self)**: Prepara el objeto para ser iterado, inicializando un **self.contador** interno a cero que servirá para identificar unívocamente el ID de cada variante generada.
- **__next__(self)**: Constituye el motor de la clase. En cada invocación ejecuta tres pasos:
 1. Utiliza **sample(self.c, self.NumPreguntas)** para obtener una lista con el número de preguntas solicitado, garantizando que en una misma variante no se repita ninguna cuestión.
 2. Incrementa el contador de variantes.

3. Devuelve una tupla con el identificador en formato de cadena, el enunciado estático y la lista aleatoria de preguntas seleccionadas con sus respectivos estados lógicos (`True/False`).

Nota de diseño: Dado que no existe una condición de parada intrínseca (como un extremo de lista), el bucle que consuma este iterador debe limitarse explícitamente desde el exterior (por ejemplo, mediante un bucle `for` con un rango acotado o controlando el número de peticiones).

5.2. Ejemplo de funcionamiento

A continuación se ilustra el comportamiento de la clase utilizando un banco de diez enunciados cotidianos relacionados con la vida académica.

```
banco = [
    ("Todo alumno que va a clase aprueba", False),
    ("Todo alumno que suspende va a clase", False),
    ("Todo alumno que sabe aprueba", True),
    ("Todo alumno que NO sabe suspende", False),
    ("Algunos alumnos aprueban copiando", True),
    ("Copiar es intolerable", True),
    ("Si aprueban todos es estupendo", True),
    ("Si aprueban todos eres buen profesor", False),
    ("Si suspenden todos eres mal profesor", False),
    ("Si suspenden todos es frustrante", True),]
```

5.2.1. Generación de cuatro variantes cortas

En este primer ensayo se configuran cuatro ejercicios, solicitando únicamente dos preguntas tipo verdadero/falso para cada uno.

```
enunciado = "Marque las verdaderas:"
p = ProblemaVF (enunciado, banco, 2)      # Dos preguntas del banco por variante
p0 = iter(p)

for i in range(4):                        # Generamos cuatro variantes y paramos
    print(next(p0))
```

```
('1', 'Marque las verdaderas:', [('Si suspenden todos es frustrante', True), ('Si aprueban todos es estupendo', True)])
('2', 'Marque las verdaderas:', [('Copiar es intolerable', True), ('Todo alumno que suspende va a clase', False)])
('3', 'Marque las verdaderas:', [('Todo alumno que suspende va a clase', False), ('Si suspenden todos eres mal profesor', False)])
('4', 'Marque las verdaderas:', [('Todo alumno que suspende va a clase', False), ('Todo alumno que sabe aprueba', True)])
```

5.2.2. Generación de cinco variantes con presentación formateada

Como la representación en una sola tupla puede dificultar la lectura a medida que el número de ítems crece, vamos a desempaquetar los componentes devueltos por el iterador para volcar los datos de manera estructurada para que se vea mejor el resultado, pues vamos a mostrar cada cuestión en una línea independiente.

Dado que ahora lo vamos a ver mejor, aumentamos la complejidad solicitando tres preguntas por variante para un total de cinco ejercicios:

```
p = ProblemaVF (enunciado, banco, 3)      # Tres preguntas por variante
p0 = iter(p)

for i in range(5):
    # Desempaquetamos la tupla en variables
    id_pregunta, EnunciadoCompleto, lista_Cuestiones = next(p0)

    # Imprimimos los primeros datos
    f"ID: {id_pregunta}"
    print(f"Enunciado completo: {EnunciadoCompleto}")

    # Imprimimos la lista de cuestiones, cada cuestión en una línea
    print(*lista_Cuestiones, sep='\n')
    print('\n')
```

Enunciado completo: Marque las verdaderas:
('Copiar es intolerable', True)
('Si suspenden todos eres mal profesor', False)
('Si suspenden todos es frustrante', True)

Enunciado completo: Marque las verdaderas:
('Todo alumno que suspende va a clase', False)
('Si aprueban todos es estupendo', True)
('Si aprueban todos eres buen profesor', False)

Enunciado completo: Marque las verdaderas:
('Si suspenden todos eres mal profesor', False)
('Copiar es intolerable', True)
('Si aprueban todos es estupendo', True)

Enunciado completo: Marque las verdaderas:
('Si suspenden todos es frustrante', True)
('Todo alumno que suspende va a clase', False)
('Todo alumno que va a clase aprueba', False)

Enunciado completo: Marque las verdaderas:
('Todo alumno que NO sabe suspende', False)
('Todo alumno que suspende va a clase', False)
('Todo alumno que sabe aprueba', True)

Como se puede apreciar en el resultado impreso, cada uno de los cinco bloques representa un ejercicio completamente autónomo con un identificador secuencial, conservando el enunciado general invariante-mente e integrando un muestreo dinámico y heterogéneo extraído directamente de la lista de afirmaciones almacenada con el nombre de **banco**.

Capítulo 6

¿Y ahora qué?

6.1. ¿Qué tenemos?

- Contamos con un módulo de cálculo proposicional que determina la veracidad o falsedad de cada enunciado, basada en los supuestos de cada caso particular.
- Poseemos una metodología para programar ejercicios utilizando listas de sublistas que incluyen textos, supuestos y cuestiones.
- Tenemos la capacidad de desempaquetar dichas listas mediante `ProblemaTipo`, lo que nos permite generar múltiples ejercicios, con la tranquilidad de que el módulo de cálculo proposicional se encarga de identificar qué opciones son verdaderas y cuáles son falsas, de acuerdo con la composición de los supuestos de cada enunciado.

Con esto, hemos desarrollado un procedimiento para generar una amplia variedad de ejercicios de opción múltiple.

6.2. ¿Qué nos falta?

Falta dar formato a lo que hemos desarrollado para ser usado con los estudiantes. El módulo `CalcProp-export` nos permitirá crear bancos de preguntas en formato $\text{\LaTeX}$, que podrán ser utilizados con [Auto Multiple Choice \(AMC\)](#), así como también generar bancos en formato `xml` para su empleo en el entorno Moodle mediante el paquete de $\text{\LaTeX}$ [moodle](#), específicamente para cuestiones de [opción múltiple](#).